Using Quantum Natural Language Processing for Sentiment Classification and Next-Word Prediction in Sentences Without Fixed Syntactic Structure

David Peral-García[1(✉)], Juan Cruz-Benito[2,3], and Francisco José García-Peñalvo[3]

[1] Faculty of Science, Universidad de Salamanca (https://ror.org/02f40zc51), Plaza de los Caídos s/n, 37008 Salamanca, Spain
daveral@usal.es

[2] IBM Quantum, IBM T.J. Watson Research Center, Yorktown Heights, NY 10598, USA
juan.cruz.benito@ibm.com

[3] GRIAL Research Group, Department of Computers and Automatics, Research Institute for Educational Sciences, Universidad de Salamanca (https://ror.org/02f40zc51), Paseo de Canalejas, 169, 37008 Salamanca, Spain
fgarcia@usal.es

Abstract. Quantum Computing is envisioned as one of the scientific areas with greater transformative potential. Already there exist applications running in quantum devices for different areas, like cybersecurity, chemistry, or machine learning. One subarea being developed under quantum machine learning is quantum natural language processing. Following the promising results existing in problems like sentiment classification or next-word prediction, this paper presents two proofs of concept to demonstrate how these two tasks can be solved using quantum computing. For the first task showcased, sentiment classification, we employ the removal of caps and cups morphisms to make the string diagrams simpler and more efficient. In the case of next-word prediction, we show how to solve the task for sentences with previously unknown syntactic structures by applying a classical Random Forest machine learning algorithm that classifies the syntactic structure and enables our QNLP algorithm to infer the proper string model.

Keywords: Quantum computing · Quantum machine learning · Quantum natural language processing

1 Introduction and State of the Art

The field of quantum computing has emerged as an active area of academic and corporate research and development in the past years. One of the reasons explaining the emergence of quantum computing is that quantum computers could solve

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
A. Lopata et al. (Eds.): ICIST 2023, CCIS 1979, pp. 235–243, 2024.
https://doi.org/10.1007/978-3-031-48981-5_19

specific problems that classical computers could not solve due to computational complexity or the time classical computers would take to resolve. The technical implementation of these quantum computers, albeit it is still not fault-tolerant, is becoming a reality while we enter into the utility era for quantum computers [13]. Companies like IBM, Google, or Rigetti are developing quantum hardware with different approaches, like superconductors, trapped ions, neutral atoms, diamond NV centers, etc. Different approaches exist in the hardware because each implementation has advantages and disadvantages, for example, different decoherence times or gate fidelity. At the same time, the companies and open source community are improving the capabilities that people take advantage of using software libraries like Qiskit [17], Q#, CirQ [8] or Pennylane [3].

The basic unit in quantum computing is the quantum bit, known as qubit. The Bra Ket notation represents it and indicates that we are referring to a quantum state. The state $|0\rangle$ represent the matrix $\begin{bmatrix}1\\0\end{bmatrix}$ and the state $|1\rangle$ represent the matrix $\begin{bmatrix}0\\1\end{bmatrix}$.

To make operations when the qubits are defined, we use quantum gates. An example of these gates are the X-Gate $\begin{bmatrix}0&1\\1&0\end{bmatrix}$, Y-Gate $\begin{bmatrix}0&-i\\i&0\end{bmatrix}$, Z-Gate $\begin{bmatrix}1&0\\0&-1\end{bmatrix}$ or H-Gate $\frac{1}{\sqrt{2}}\begin{bmatrix}1&1\\1&-1\end{bmatrix}$.



(a) Z Gate (b) H Gate

Fig. 1. Z Gate applied to a $|0\rangle$ state and H Gate applied to a $|0\rangle$ state

Algorithms theoretically demonstrated in the last century now have a real-world implementation in actual quantum computers. For example, Grover's search algorithm [11], Shor's algorithm [20] for integer factorization in polynomial time or superdense coding, a protocol to communicate a number of classical bits of information by transmitting a smaller amount of qubits (see Fig. 1).

Moreover, quantum computing fields of application have been expanded to other areas like cybersecurity [2,18,24], physics [1,16] or machine learning [4,19] (see Fig. 2), a review of quantum machine learning algorithms and their applications can be found here [10]. One of the subareas of QML, quantum natural language processing QNLP, has been promising studies like Quantum Vision Transformers [5] (see Fig. 3) or the creation of the DisCoPy [9], DisCoCirc [6] and lambeq [12] frameworks. These frameworks already enabled applications in

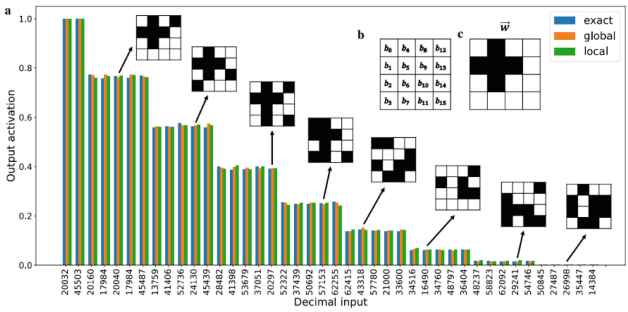


Fig. 2. Classification of 4×4 pixel images [21]

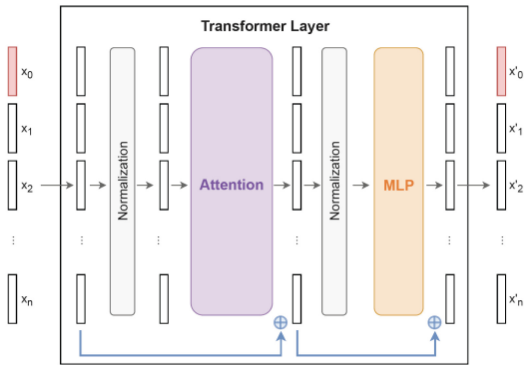


Fig. 3. Quantum Vision Transformer - Single Transformer Layer [5]

fields like word prediction [22], sentiment classification, or interpretable generative music systems [15].

Referring to the DisCoPy-DisCoCirc framework, one of its main aspects is the use of ZX Calculus [7], a graphical calculus for reasoning about quantum phenomena. A ZX-diagram is a graphical representation of a linear map between qubits reminiscent of a conventional quantum circuit diagram [23].

Building on the recent proposal of quantum algorithms for NLP tasks by Zeng and Coecke [48], we take advantage of the tensor structure in order to construct a map from DisCoCat models to variational quantum circuits, where ansatzes corresponding to lexical categories are connected according to the grammar to form circuits for arbitrary syntactic units.

As the authors did in [14], we will try to connect the lexical categories of the sentence with their ansatzes according to the grammar to form circuits for arbitrary syntactic units Fig. 4. This paper aims to achieve classical NLP tasks that

are already solved, like word prediction or sentiment classification with quantum computing, using the DisCoPy-DisCoCirc-lambeq frameworks while introducing some improvements like using classical machine learning algorithms to classify the syntactic structure of a sentence before inferring the string diagram.

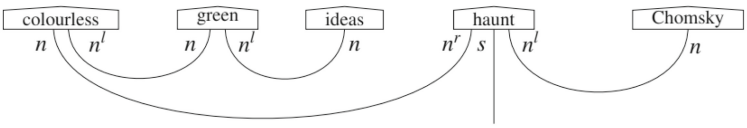


Fig. 4. String Diagram [14]

2 QNLP Proofs of Concept

Using the DisCoPy-lambeq framework cited in the previous section, DisCoPy, we will present two proofs of concept for different NLP tasks: sentiment classification and next-word prediction in sentences with previously unknown syntactic structure.

2.1 QNLP for Sentiment Classification

In this example, we will test classification tasks in different sentences, trying to predict positive or negative sentiment. We present the modeling process, which includes the codification and transformation of the data and model training.

First, the input sentence is encoded into a string diagram using a parser, for example, DepCGG or BobCat parser (see Fig. 5). Second, we removed the cups from the string diagram (see Fig. 6). For the sake of simplifying the string diagram and making it more efficient, we remove the unnecessary cups and caps. The cups and caps are morphisms able to wire an object and its adjoint. Third, we parameterized the diagram and transformed it into a quantum circuit using an ansatz. Some examples of the ansatzes that lambeq provides are SpiderAnsatz (see Fig. 7), IQPAnsatz, Sim14Ansatz (see Fig. 8), Sim15Ansatz or StronglyEntanglementAnsatz. Fourth, when the parameterisable circuit is created, a compatible backend with the model must be defined. We can select a classical device, a quantum simulator, or a real quantum device; when this option is selected, we train the model in the selected device.

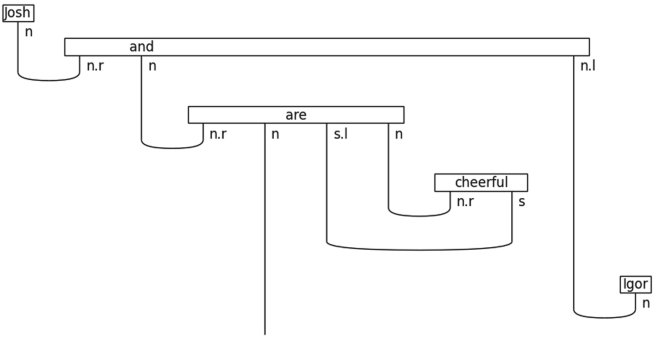


Fig. 5. String Diagram

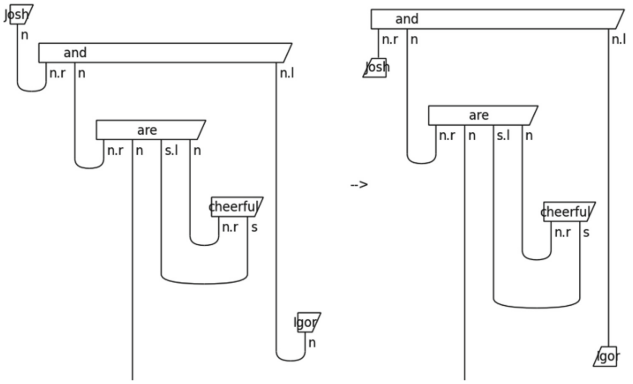


Fig. 6. String Diagram - Remove Cups

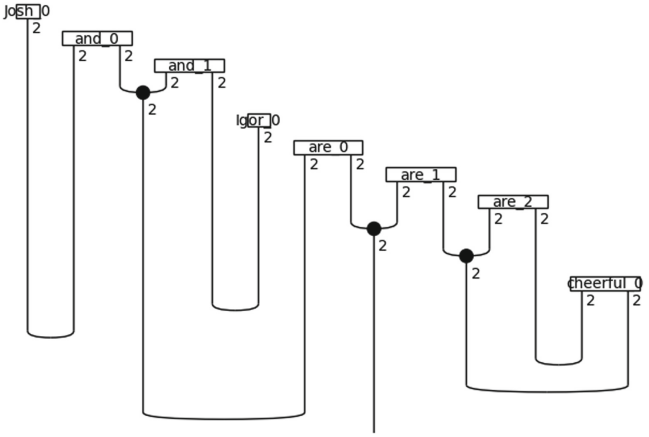


Fig. 7. Spider Ansatz

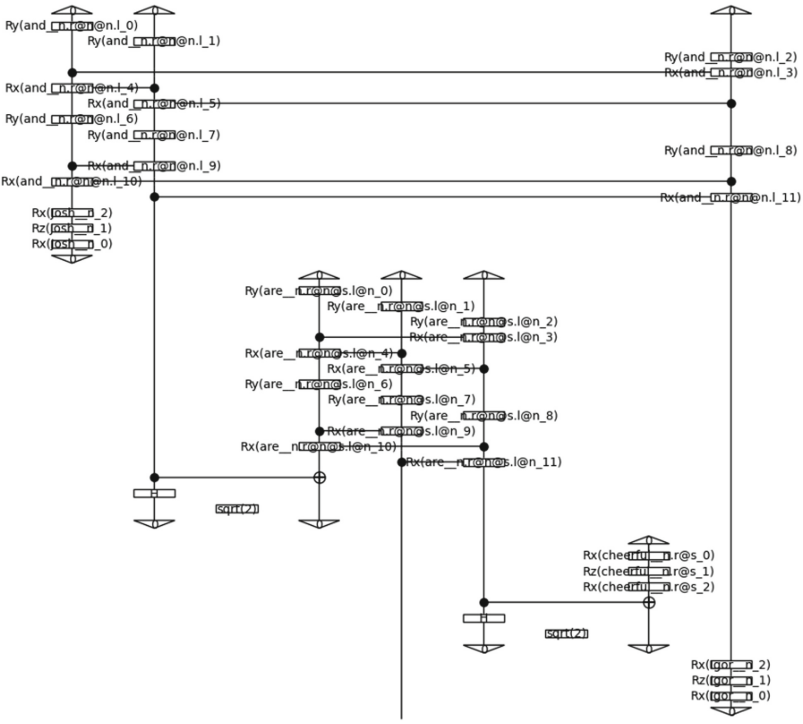


Fig. 8. Sim14 Ansatz

2.2 QNLP for Next Word Prediction in Sentences with Previously Unknown Syntactic Structure

In the second proof of concept, we try to predict a missing word in a sentence with an unknown syntactic structure. In the first part of the algorithm, we train

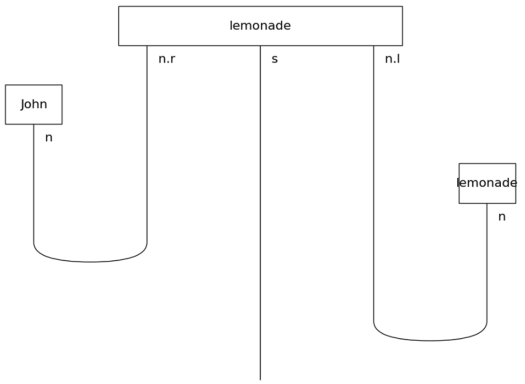


Fig. 9. Prediction Model Input Sentence

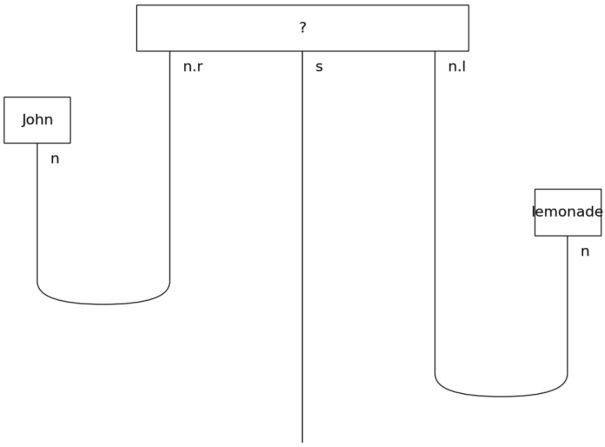


Fig. 10. Prediction Model Missing Value

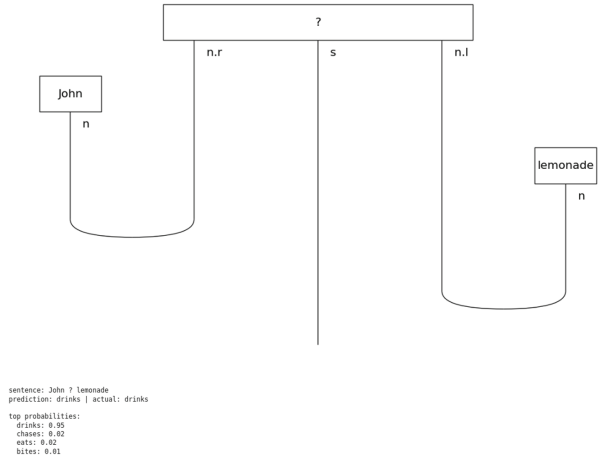


Fig. 11. Prediction Model Results

a Random Forest model to obtain the syntactic structure of a sentence with a missing word. When we have predicted the syntactic structure, we infer it in the DisCoPy string model (see Fig. 10). After converting the sentence into a string diagram, we parameterize it into a model and train it (see Fig. 11).

3 Conclusions and Future Lines

As discussed in the first section, quantum computing is an area with great potential showing promising results in different areas. Regarding QNLP, the conceptual and mathematical basis of this subfield has been established, the creation

of hybrid models, models which can run in a classical or quantum device, allow us great compatibility in the way we develop the first part of the algorithm, the encoding of the sentences into string diagrams (see Fig. 9).

We can compute the sentiment classification task in a quantum device using the Discopy-lambeq framework through the classification proofs of concept. First, we transform the sentence into a string diagram. Second, we convert this diagram into an ansatz. Finally, we infer this ansatz into a model and run it in a quantum device or a quantum simulator. As a next step, we can test different ansatz to maximize the model's accuracy. Also, we can increase the number of words that the model can predict, which is the number of words in the whole sentence, and increase the complexity of the sentence with complex syntax and more words, which affects both models.

Regarding the prediction task, first, we use a Random Forest classical model to determine the correct syntactic structure of the sentence. After that, we parse the sentence into a diagram. When this step is done, we train the model with the unknown word and a defined grammatical structure. We intend to use a quantum model syntax classifier to improve the algorithm's first part of the next-word prediction task. For example, we are replacing the classical Random Forest model with a quantum classification model. Our plan for the second step of the algorithm is to update the weights using a QNN.

References

1. Bauer, C.W., Freytsis, M., Nachman, B.: Simulating collider physics on quantum computers using effective field theories (2021)
2. Bennett, C.H., Brassard, G.: Quantum cryptography: public key distribution and coin tossing. Theor. Comput. Sci. **560**, 7–11 (2014). https://doi.org/10.1016/j.tcs.2014.05.025
3. Bergholm, V., et al.: Pennylane: automatic differentiation of hybrid quantum-classical computations (2022)
4. Biamonte, J., Wittek, P., Pancotti, N., Rebentrost, P., Wiebe, N., Lloyd, S.: Quantum machine learning. Nature **549**(7671), 195–202 (2017). https://doi.org/10.1038/nature23474
5. Cherrat, E.A., Kerenidis, I., Mathur, N., Landman, J., Strahm, M., Li, Y.Y.: Quantum vision transformers (2022)
6. Coecke, B.: The mathematics of text structure (2020)
7. Coecke, B., Duncan, R.: Interacting quantum observables: categorical algebra and diagrammatics. New J. Phys. **13**(4), 043016 (2011). https://doi.org/10.1088/1367-2630/13/4/043016
8. Developers, C.: Cirq (2022). https://doi.org/10.5281/zenodo.7465577. See full list of authors on Github: https://github.com/quantumlib/Cirq/graphs/contributors
9. de Felice, G., Toumi, A., Coecke, B.: DisCoPy: monoidal categories in Python. Electr. Proc. Theor. Comput. Sci. **333**, 183–197 (2021). https://doi.org/10.4204/eptcs.333.13
10. García, D.P., Cruz-Benito, J., García-Peñalvo, F.J.: Systematic literature review: quantum machine learning and its applications (2022)

11. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing, STOC '96, pp. 212–219. Association for Computing Machinery, New York (1996). https://doi.org/10.1145/237814.237866
12. Kartsaklis, D., et al.: Lambeq: an efficient high-level python library for quantum NLP. arXiv preprint: arXiv:2110.04236 (2021)
13. Kim, Y., et al.: Evidence for the utility of quantum computing before fault tolerance. Nature **618**, 500–505 (2023). https://doi.org/10.1038/s41586-023-06096-3
14. Meichanetzidis, K., Gogioso, S., de Felice, G., Chiappori, N., Toumi, A., Coecke, B.: Quantum natural language processing on near-term quantum computers. Electr. Proc. Theor. Comput. Sci. **340**, 213–229 (2021). https://doi.org/10.4204/eptcs.340.11
15. Miranda, E.R., Yeung, R., Pearson, A., Meichanetzidis, K., Coecke, B.: A quantum natural language processing approach to musical intelligence (2021)
16. Nachman, B., Provasoli, D., de Jong, W.A., Bauer, C.W.: Quantum algorithm for high energy physics simulations. Phys. Rev. Lett. **126**(6), 062001 (2021). https://doi.org/10.1103/physrevlett.126.062001
17. Qiskit contributors: Qiskit: an open-source framework for quantum computing (2023). https://doi.org/10.5281/zenodo.2573505
18. Scarani, V., Bechmann-Pasquinucci, H., Cerf, N.J., Dušek, M., Lütkenhaus, N., Peev, M.: The security of practical quantum key distribution. Rev. Modern Phys. **81**(3), 1301–1350 (2009). https://doi.org/10.1103/revmodphys.81.1301
19. Schuld, M., Sinayskiy, I., Petruccione, F.: An introduction to quantum machine learning. Contemp. Phys. **56**(2), 172–185 (2014). https://doi.org/10.1080/00107514.2014.964942
20. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM J. Comput. **26**(5), 1484–1509 (1997). https://doi.org/10.1137/s0097539795293172
21. Tacchino, F., Barkoutsos, P., Macchiavello, C., Tavernelli, I., Gerace, D., Bajoni, D.: Quantum implementation of an artificial feed-forward neural network. Quantum Sci. Technol. **5**(4) (2020). https://doi.org/10.1088/2058-9565/abb8e4. arXiv:1912.12486
22. Toumi, A., Koziell-Pipe, A.: Functorial language models (2021)
23. van de Wetering, J.: ZX-calculus for the working quantum computer scientist (2020)
24. Zhang, W., Ding, D.S., Sheng, Y.B., Zhou, L., Shi, B.S., Guo, G.C.: Quantum secure direct communication with quantum memory. Phys. Rev. Lett. **118**(22), 220501 (2017). https://doi.org/10.1103/physrevlett.118.220501